

HostelFix: Technical Project and Oral-Defense Document

- [HostelFix](#)
 - [Design and Implementation of a Role-Based Maintenance Reporting and Tracking System for University of Ghana Halls](#)
 - [Abstract](#)
 - [Table of Contents](#)
- [1. Introduction](#)
- [2. Problem Definition and Motivation](#)
 - [2.1 Problem statement](#)
 - [2.2 Evidence and validation status](#)
 - [2.3 Existing alternatives and their gaps](#)
 - [2.4 Why this is a computing problem](#)
 - [2.5 Novelty statement](#)
- [3. Stakeholders and Scope](#)
 - [3.1 Stakeholders](#)
 - [Student resident](#)
 - [Hall manager or caretaker](#)
 - [Maintenance personnel](#)
 - [University administrator](#)
 - [System administrator](#)
 - [Indirect stakeholders](#)
 - [3.2 In scope](#)
 - [3.3 Out of scope or incomplete](#)
 - [3.4 Residence scope discrepancy discovered during audit](#)
- [4. Requirements Engineering](#)
 - [4.1 Elicitation approach](#)
 - [4.2 Functional requirements](#)
 - [4.3 Non-functional requirements](#)
 - [4.4 Requirements traceability examples](#)
 - [4.5 Handling conflicting requirements](#)
- [5. Operational Workflow](#)
 - [5.1 Main lifecycle](#)
 - [5.2 Detailed flow](#)
 - [5.3 Direct resolution](#)
 - [5.4 Maintenance-role justification](#)
 - [5.5 Emergency behavior](#)
- [6. System Architecture](#)
 - [6.1 Architectural style](#)

- [6.2 Why a modular monolith](#)
- [6.3 Alternatives rejected](#)
 - [Microservices](#)
 - [Next.js-only backend](#)
 - [Serverless functions](#)
- [6.4 Internal module pattern](#)
- [6.5 Communication protocols](#)
- [6.6 Single points of failure](#)
- [7. Technology and Design Decisions](#)
 - [7.1 Frontend](#)
 - [7.2 Backend](#)
 - [7.3 Why MongoDB](#)
 - [7.4 Why external image storage](#)
- [8. Data Design](#)
 - [8.1 Main collections](#)
 - [Users](#)
 - [Halls and locations](#)
 - [Issues](#)
 - [Issue events](#)
 - [Audit logs](#)
 - [Refresh and reset tokens](#)
 - [Reference counters](#)
 - [8.2 Relationship overview](#)
 - [8.3 Indexes](#)
 - [8.4 Concurrent writes](#)
 - [8.5 Normalization and denormalization](#)
 - [8.6 Growth to 100 times the data](#)
- [9. API and Module Design](#)
 - [9.1 API base and documentation](#)
 - [9.2 Important endpoint groups](#)
 - [Authentication](#)
 - [Issues](#)
 - [Residence directory](#)
 - [Administration](#)
 - [9.3 Request validation](#)
 - [9.4 Error format](#)
 - [9.5 Current contract limitation](#)
- [10. Frontend Design](#)
 - [10.1 Shared role-based route](#)
 - [10.2 Role views](#)
 - [Student](#)
 - [Hall manager](#)
 - [Maintenance](#)

- [Administrator](#)
 - [10.3 Session guard](#)
 - [10.4 State management](#)
- [11. Algorithms and Complexity](#)
 - [11.1 State-transition validation](#)
 - [11.2 Unique reference allocation](#)
 - [11.3 Role-scoped issue retrieval](#)
 - [11.4 Overdue calculation](#)
 - [11.5 Auto-close job](#)
 - [11.6 Analytics](#)
 - [11.7 Performance evidence](#)
- [12. Security and Governance](#)
 - [12.1 Authentication](#)
 - [12.2 Authorization](#)
 - [12.3 Token theft response](#)
 - [12.4 Threat model: leading attack vectors](#)
 - [Broken access control](#)
 - [Credential attacks](#)
 - [Injection and malicious input](#)
 - [Malicious or privacy-invasive images](#)
 - [12.5 OWASP alignment](#)
 - [12.6 Data at rest and in transit](#)
 - [12.7 Database credential leakage](#)
 - [12.8 Governance](#)
- [13. Software Engineering Process](#)
 - [13.1 Development approach](#)
 - [13.2 Code organization](#)
 - [13.3 Version control](#)
 - [13.4 Static analysis and quality gates](#)
- [14. Testing and Evaluation](#)
 - [14.1 Testing strategy](#)
 - [Unit testing](#)
 - [Integration testing](#)
 - [Permission testing](#)
 - [Frontend testing](#)
 - [14.2 Current verified results](#)
 - [14.3 Example failure and correction](#)
 - [14.4 Correctness versus “it runs”](#)
 - [14.5 Missing evaluation](#)
- [15. Deployment and Operations](#)
 - [15.1 Local runtime](#)
 - [15.2 Production topology](#)
 - [15.3 Scaling to ten times the load](#)

- [16. Limitations and Future Work](#)
 - [16.1 Biggest current limitation](#)
 - [16.2 Important technical limitations](#)
 - [16.3 Prioritized roadmap](#)
 - [Immediate correctness](#)
 - [Operational completeness](#)
 - [Production readiness](#)
 - [Research and evaluation](#)
- [17. Critical Reflection and Intellectual Contribution](#)
 - [17.1 What would be done differently](#)
 - [17.2 Most defensible design decision](#)
 - [17.3 Least certain design decision](#)
 - [17.4 Intellectual contribution](#)
- [18. Oral-Defense Question Bank and Model Answers](#)
 - [18.1 Problem Definition and Motivation](#)
 - [Q: What specific problem does this solve, and what evidence shows it is real?](#)
 - [Q: Who are the users and how was the problem validated?](#)
 - [Q: What existing solutions were reviewed, and what do they fail to do?](#)
 - [Q: Why is this a computer science or IT problem?](#)
 - [Q: State the novelty in one sentence.](#)
 - [18.2 Requirements Engineering](#)
 - [Q: How were requirements derived?](#)
 - [Q: What was deliberately scoped out?](#)
 - [Q: How do requirements trace to deliverables?](#)
 - [Q: How would you handle stakeholder/technical conflict?](#)
 - [18.3 System Design and Architecture](#)
 - [Q: Walk through the architecture.](#)
 - [Q: Why a modular monolith rather than microservices?](#)
 - [Q: What are the single points of failure?](#)
 - [Q: How do components communicate and why REST?](#)
 - [Q: What patterns were used?](#)
 - [18.4 Data Design](#)
 - [Q: Explain the schema.](#)
 - [Q: What keys and indexes matter?](#)
 - [Q: How are integrity and concurrency handled?](#)
 - [Q: What breaks at 100 times the data?](#)
 - [Q: Why NoSQL?](#)
 - [18.5 Algorithms and Complexity](#)
 - [Q: What are the core algorithms?](#)
 - [Q: Why atomic counter generation?](#)
 - [Q: Where is the performance bottleneck and how do you know?](#)
 - [Q: What would you optimize first?](#)
 - [18.6 Security and Governance](#)

- [Q: How do authentication and authorization work?](#)
- [Q: What if a token is stolen?](#)
- [Q: What are the top three attacks?](#)
- [Q: How is data protected?](#)
- [Q: Which standard was followed?](#)
- [Q: What if database credentials leak?](#)
- [18.7 Software Engineering Process](#)
 - [Q: What methodology was followed?](#)
 - [Q: How was version control managed?](#)
 - [Q: What testing was performed?](#)
 - [Q: Was there code review or CI/CD?](#)
 - [Q: How is maintainability supported?](#)
- [18.8 AI/ML Questions](#)
 - [Q: What AI model did the system use?](#)
 - [Q: How would you evaluate a future model?](#)
- [18.9 Testing and Evaluation](#)
 - [Q: How do you know the system meets requirements?](#)
 - [Q: Give an initially failing case and fix.](#)
 - [Q: Was load testing performed?](#)
 - [Q: Who tested it besides the developer?](#)
- [18.10 Critical Reflection](#)
 - [Q: What is the single biggest limitation?](#)
 - [Q: What would you do differently?](#)
 - [Q: What did you learn technically?](#)
 - [Q: Defend a decision you are least confident about.](#)
 - [Q: What did you add rather than assemble?](#)
 - [Q: How does this demonstrate CS/IT mastery?](#)
- [19. Demonstration Script](#)
 - [19.1 Preparation](#)
 - [19.2 Main scenario](#)
 - [19.3 Security scenario](#)
 - [19.4 Reflection during demonstration](#)
- [20. Conclusion](#)
- [21. References and Evidence Map](#)
 - [21.1 Internal project evidence](#)
 - [21.2 External frameworks to cite in a formal academic bibliography.](#)
 - [Appendix A: Short Defense Opening](#)
 - [Appendix B: One-Minute Architecture Answer](#)
 - [Appendix C: One-Minute Limitation Answer](#)

HostelFix

Design and Implementation of a Role-Based Maintenance Reporting and Tracking System for University of Ghana Halls

Technical Project and Oral-Defense Document

Candidate: [Insert candidate name]

Student ID: [Insert student ID]

Programme: [Insert programme]

Department: [Insert department]

Supervisor: [Insert supervisor name]

Institution: University of Ghana

Date: September 2026

Abstract

HostelFix is a web-based maintenance reporting and tracking system designed for University of Ghana student halls. It provides a shared operational record for students, hall managers, maintenance personnel, university administrators, and system administrators. The system addresses fragmented reporting through verbal complaints, paper logbooks, informal messaging, and disconnected follow-up. Its purpose is not to replace the physical and social processes through which maintenance work occurs. Instead, it records four operational facts: what is broken, whether the responsible team knows, whether the problem was fixed, and whether the resident disputes the resolution.

The implemented system uses a Next.js 16 frontend, an Express 5 and TypeScript backend, and MongoDB through Mongoose. Authentication uses short-lived JSON Web Tokens and opaque refresh tokens stored in HTTP-only cookies. Authorization combines role-based access control with object-level ownership and assigned-hall checks. Images are uploaded directly to Cloudinary, while MongoDB stores controlled metadata and image URLs. Material actions produce issue events and audit logs. Background jobs handle notification delivery, overdue escalation, and automatic closure of resolved issues after a configurable dispute window.

A major design contribution is the simplification of the issue lifecycle from a rigid six-stage digital workflow to a lightweight operational record:

```
Submitted -> Acknowledged -> Resolved -> Closed
      |                               |
      Rejected                       Reopened
```

Acknowledgement records awareness rather than granting permission for work. Individual assignment and an `in_progress` update are optional, allowing maintenance to proceed through practical channels such as direct conversation, telephone, WhatsApp, a caretaker, or a contractor.

This design reduces administrative bottlenecks while retaining accountability, scoped access, analytics, and an immutable history.

The backend currently passes 30 automated tests covering unit, integration, and permission-denial behavior. The frontend passes its available unit tests, TypeScript checking, linting with non-blocking image warnings, and a production build. Important limitations remain: formal user-acceptance and load testing have not been documented; duplicate reports cannot be linked; resolution messages are accepted by validation but are not currently persisted; completion evidence is not separately modeled; and production infrastructure such as persistent queues, centralized monitoring, backup policy, and emergency-contact UI still requires completion.

Keywords: maintenance reporting, university halls, role-based access control, MongoDB, Next.js, Express, workflow design, audit trail, information systems

Table of Contents

1. Introduction
 2. Problem Definition and Motivation
 3. Stakeholders and Scope
 4. Requirements Engineering
 5. Operational Workflow
 6. System Architecture
 7. Technology and Design Decisions
 8. Data Design
 9. API and Module Design
 10. Frontend Design
 11. Algorithms and Complexity
 12. Security and Governance
 13. Software Engineering Process
 14. Testing and Evaluation
 15. Deployment and Operations
 16. Limitations and Future Work
 17. Critical Reflection and Intellectual Contribution
 18. Oral-Defense Question Bank and Model Answers
 19. Demonstration Script
 20. Conclusion
 21. References and Evidence Map
-

1. Introduction

Maintenance reporting in a university residence is both a technical and organizational problem. A student may notice a leaking pipe, damaged electrical fitting, blocked washroom, or structural defect. The problem may be reported verbally to a porter, through a hall messaging group, by telephone, or in a paper logbook. Work may then be assigned informally to a plumber, electrician, caretaker, university maintenance employee, or external contractor. The physical work can occur before any formal digital update is made.

A badly designed information system attempts to control every one of these actions. It may require verification, assignment, acceptance, and progress updates before a worker is allowed to record a repair. That approach creates a bottleneck and encourages staff to bypass the application. HostelFix therefore treats the application as a reliable operational record, not as the exclusive channel through which work must happen.

The project delivers:

- A responsive web interface for students and staff.
- A REST API containing domain and authorization rules.
- A MongoDB data store for current state and historical events.
- Role-aware dashboards under one canonical `/dashboard` route.
- Controlled issue creation, acknowledgement, resolution, reopening, and closure.
- Hall-scoped access for hall managers and maintenance personnel.
- Cross-hall analytics and audit access for administrators.
- Background processing for notifications, overdue escalation, and auto-close.
- OpenAPI documentation and automated tests.

2. Problem Definition and Motivation

2.1 Problem statement

Maintenance reports in student halls can become fragmented across verbal reports, messaging applications, paper records, and personal follow-up. This produces several failures:

1. A student cannot reliably determine whether a report was received.
2. Hall management lacks a consolidated queue of unresolved problems.
3. Maintenance completion may not be communicated back to the reporter.
4. Repeated faults and unsuccessful repairs are difficult to identify.
5. University administrators lack comparable cross-hall response data.
6. Informal maintenance work leaves a weak accountability trail.

HostelFix solves the information-coordination part of this problem by maintaining a role-scoped, auditable record without requiring every physical handoff to occur digitally.

2.2 Evidence and validation status

The strongest evidence currently represented in the project is operational reasoning captured in the accepted workflow decision: hall maintenance often proceeds through caretakers, direct conversation, telephone, messaging, and existing maintenance structures. The original workflow was explicitly rejected because it would force digital updates before work could proceed.

However, the repository does not contain a completed survey dataset, interview transcripts, a signed user-acceptance report, or quantified baseline measurements from the halls. Therefore, the academically defensible statement is:

The problem is supported by observed and domain-informed workflow analysis, but formal empirical validation with representative University of Ghana stakeholders remains future evaluation work.

A defense should not claim surveys or interviews unless the candidate can produce them. A suitable validation plan would involve:

- Semi-structured interviews with hall managers, porters, maintenance staff, and students.
- A questionnaire measuring current reporting channels, lost reports, response visibility, and satisfaction.
- Review of existing maintenance logbooks or messaging records, subject to approval and privacy controls.
- A pilot in one hall comparing report acknowledgement time, resolution time, and reopening rate before and after adoption.

2.3 Existing alternatives and their gaps

The system is not claiming that no maintenance tools exist. The relevant alternatives are:

Alternative	Strength	Gap addressed by HostelFix
Verbal report	Fast and familiar	Weak traceability, follow-up, and analytics
WhatsApp or telephone	Immediate communication	Reports are mixed with unrelated conversation and difficult to aggregate
Paper logbook	Simple shared record	Limited remote access, search, notifications, and cross-hall analysis
Generic ticketing system	Mature ticket management	Often imposes assignment-heavy workflows and is not

Alternative	Strength	Gap addressed by HostelFix
		hall-, room-, or student-aware
Spreadsheet	Flexible and inexpensive	Weak access control, concurrency management, auditability, and user-specific views

HostelFix combines the speed of informal coordination with a controlled operational record designed around halls, locations, resident ownership, dispute windows, and University roles.

2.4 Why this is a computing problem

The underlying maintenance shortage or procurement process cannot be solved by software alone. The computing problem is the reliable management of identity, access, state, history, notifications, and analytics across multiple actors. Specifically, the project requires:

- Authentication and secure session management.
- Object-level authorization over sensitive residence records.
- Concurrent creation of unique report references.
- State-transition enforcement.
- Query optimization and indexing.
- Background scheduling and asynchronous notification.
- Data validation and integrity controls.
- Responsive role-based user interfaces.
- Auditability and measurable operational outcomes.

Thus, HostelFix is an information-system intervention inside a broader organizational process. It does not claim that software itself repairs facilities.

2.5 Novelty statement

HostelFix contributes a low-friction, hall-scoped maintenance record that preserves accountability while deliberately allowing physical work to proceed outside the digital workflow.

3. Stakeholders and Scope

3.1 Stakeholders

Student resident

- Reports a defect.
- Sees only their own issue records.
- Tracks acknowledgement and resolution.
- Reopens an unsuccessful resolution during the dispute window.

Hall manager or caretaker

- Sees issues only in assigned halls.
- Acknowledges, rejects, or requests clarification.
- Reviews operational priority.
- Coordinates work through practical offline or online channels.
- Can record resolution or close a resolved issue.

Maintenance personnel

- Sees issues in assigned halls.
- Can record resolution without being individually assigned.
- Does not have to mark an issue `in_progress` .

University administrator

- Sees cross-hall analytics.
- Reviews audit records and users according to policy.
- Monitors backlog, overdue reports, recurring categories, and resolution times.

System administrator

- Manages technical configuration and role changes.
- Does not rewrite historical issue events.

Indirect stakeholders

- University management.
- Porters and caretakers.
- External contractors.
- Data protection and IT operations personnel.

3.2 In scope

- Authenticated reporting by provisioned users.
- University-managed hall and location directory.

- Categories, priority, descriptions, and initial images.
- Role-aware dashboard and navigation.
- Issue status and event history.
- Hall-scoped maintenance and management access.
- Optional assignment metadata.
- Notifications and audit logging.
- Analytics and overdue calculation.
- Auto-close and reopening.
- OpenAPI documentation.

3.3 Out of scope or incomplete

- Public registration.
- Procurement, inventory, vendor payment, and spare-parts management.
- GPS technician tracking.
- Artificial intelligence or machine-learning diagnosis.
- Real-time emergency dispatch.
- Formal contractor onboarding.
- Structured duplicate-report linking.
- Separate completion-photo evidence.
- Production-grade persistent queues and multi-instance rate-limit storage.
- A documented institution-wide user-acceptance or load test.

3.4 Residence scope discrepancy discovered during audit

The original documentation described nine halls: five traditional halls and four UGEL halls. The current seed implementation contains ten halls because it also includes Diamond Jubilee Hall. This must be resolved before final institutional deployment: either formally approve the tenth hall and update all scope statements, or remove it from the approved seed list. This document reports the current code truth rather than hiding the discrepancy.

4. Requirements Engineering

4.1 Elicitation approach

The requirements were derived primarily through:

- Stakeholder-role analysis.
- Use-case analysis.
- Workflow modeling.
- Security and permission-matrix analysis.
- Iterative architectural review.

- Critical reflection on real maintenance practices.

The pivotal requirement change came from questioning the original rigid lifecycle. The system initially modeled Submitted → Verified → Assigned → In Progress → Resolved → Closed . Operational analysis showed that this would make software a prerequisite for work and reduce adoption. The revised requirement was that the system must record awareness and completion without blocking informal work.

No formal interview or questionnaire artifact is currently stored in the repository. That is a research-method limitation and should be stated honestly.

4.2 Functional requirements

ID	Requirement	Implemented evidence
FR-01	Provisioned users shall log in using ID/email and PIN/password	Auth module and login page
FR-02	The system shall render role-specific content under /dashboard	Auth provider, role dashboard, navigation
FR-03	A user shall create a report with hall, room, category, priority, description, and optional images	Issue form and POST /issues
FR-04	Students shall see only their own issues	Issue query policy and permission test
FR-05	Hall staff shall see only assigned-hall issues	Hall-scoped query and permission test
FR-06	Hall management shall acknowledge, reject, or request clarification	Acknowledge endpoint
FR-07	Authorized hall staff shall resolve without formal assignment	Status policy
FR-08	A resolved issue shall have a configurable dispute window	Resolution service and environment configuration
FR-09	A student shall reopen their own resolved issue	Reopen endpoint and frontend action

ID	Requirement	Implemented evidence
FR-10	The system shall auto-close expired resolved issues	Hourly cron job
FR-11	Material issue actions shall create historical events and audit logs	Issue event and audit modules
FR-12	Administrators shall view aggregate metrics	Analytics module and admin dashboard
FR-13	Images shall be uploaded externally and referenced by URL	Cloudinary signature and URL validation
FR-14	Legacy role-specific URLs shall redirect to the canonical dashboard namespace	Next.js redirects

4.3 Non-functional requirements

Category	Requirement	Current response
Security	Enforce authentication, role, ownership, and hall scope	JWT/cookies, policies, negative tests
Performance	Support indexed dashboard and queue queries	Compound MongoDB indexes and aggregation
Reliability	Save reports independently of notification-provider availability	In-memory asynchronous queue
Maintainability	Separate routes, validation, services, policies, and models	Modular-monolith structure
Usability	Minimize required status updates	Simplified lifecycle
Auditability	Preserve material actions	Issue events and audit logs
Portability	Permit future mobile or alternate clients	REST API owns domain rules
Documentation	Define API contract	OpenAPI YAML and supporting docs
Accessibility	Provide labeled, responsive interfaces	Design goal; formal WCAG audit not yet documented

4.4 Requirements traceability examples

- **Student isolation** maps to the issue list query, issue object policy, `/dashboard/issues` , and `student-isolation.test.ts` .
- **Hall isolation** maps to assigned hall IDs, hall policy, hall-scoped issue query, hall-manager dashboard, and `hall-scope.test.ts` .
- **No maintenance bottleneck** maps to the simplified constants, transition policy, resolution UI, and ADR 001.
- **Automatic closure** maps to `RESOLVE_DISPUTE_WINDOW_HOURS` , `disputeWindowExpiresAt` , the auto-close job, and auto-close integration tests.

4.5 Handling conflicting requirements

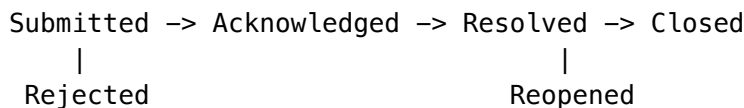
A stakeholder may ask for maximum accountability through mandatory assignment while operational users ask for fewer updates. The resolution used here is to separate mandatory state from optional metadata:

- The required state remains small.
- Assignment remains available for accountability.
- Assignment does not prevent work or resolution.
- Audit events preserve who performed the digital action.

This is a general requirements-engineering principle: preserve the underlying goal while reducing the mechanism's operational cost.

5. Operational Workflow

5.1 Main lifecycle



Clarification is an event that retains `submitted` ; it is not another persistent status.

5.2 Detailed flow

1. A student signs in and reports a defect.
2. The backend derives the reporter from the authenticated session.
3. The backend resolves the selected hall and room to a controlled location.
4. A unique reference number is allocated.
5. The issue is stored with status `submitted` .

6. A creation event and audit log are written.
7. Hall management may acknowledge, reject, or request clarification.
8. Physical work proceeds through any practical authorized channel.
9. An assigned-hall manager, maintenance user, or administrator records `resolved`.
10. The backend records `resolvedAt` and calculates `disputeWindowExpiresAt`.
11. The reporter is notified.
12. The reporter may reopen the report if the repair is unsatisfactory.
13. Otherwise, an hourly job changes expired resolved issues to `closed`.

5.3 Direct resolution

The policy allows `submitted` → `resolved`. This is deliberate. It handles cases where a caretaker or technician repairs the problem before hall management opens the application. Requiring retrospective acknowledgement would add no operational value.

5.4 Maintenance-role justification

The maintenance function is necessary, but a mandatory maintenance-system interaction is not. Keeping the role provides:

- A scoped work view for regular university personnel.
- Attribution for staff who choose to update the system.
- Separation between coordination and physical execution.

Making it optional prevents adoption failure when work is performed by a contractor, caretaker, or worker without an account. In such a case, hall management records the digital resolution.

5.5 Emergency behavior

HostelFix is not an emergency-dispatch platform. For fire, exposed wiring, severe flooding, sewage overflow, or structural danger, users must contact the responsible emergency or hall personnel first. The record can be created or updated during or after the response. A future interface should show approved emergency contacts prominently.

6. System Architecture

6.1 Architectural style

HostelFix is a client-server system with a modular-monolith backend.

Browser

|

```
| HTTPS/JSON + HTTP-only cookies
v
Next.js frontend
|
| REST/JSON with credentials
v
Express modular API
|-- Authentication and authorization
|-- Residence directory
|-- Issues and assignments
|-- Notifications and jobs
|-- Analytics and audit logs
|
+--> MongoDB
+--> Cloudinary
+--> Email adapter
```

6.2 Why a modular monolith

A modular monolith was selected because the project has one coherent domain, one primary development team, and no demonstrated need for independent service scaling. Compared with microservices, it provides:

- Simpler transactions and data consistency.
- Easier local development and deployment.
- Fewer network failure modes.
- Centralized authentication and policy enforcement.
- Lower operational cost.

The modules still create boundaries that could support later extraction. Notifications, analytics, and image processing are the best future candidates if load or team ownership justifies separation.

6.3 Alternatives rejected

Microservices

Rejected for the initial release because they would add distributed tracing, service authentication, deployment coordination, message delivery, and consistency complexity without evidence that the scale requires it.

Next.js-only backend

Rejected because domain rules should remain reusable by a future mobile client or institutional integration. Express provides an explicit API boundary.

Serverless functions

Not selected because scheduled jobs, shared authorization, local MongoDB development, and predictable modular routing are simpler in the current long-running backend. Serverless remains possible after adapting jobs and connection management.

6.4 Internal module pattern

Each backend feature generally contains:

- Routes: HTTP endpoint mapping and middleware.
- Validation: Zod request schemas.
- Controller: translation between HTTP and service calls.
- Service: business rules and persistence coordination.
- Model: Mongoose schema and indexes.
- Policy: reusable authorization decisions where applicable.

This resembles layered architecture and the service/policy patterns. Controllers remain thin, services own workflows, and policies answer permission questions.

6.5 Communication protocols

REST over HTTP with JSON was chosen because:

- Browser and mobile clients support it directly.
- The operations are resource-oriented.
- OpenAPI can document the contract.
- The expected traffic does not require binary RPC optimization.
- Debugging and institutional integration are straightforward.

An in-memory queue decouples notification attempts from report persistence. This queue is suitable for development and a single process, but production should use a durable queue such as BullMQ with Redis or a managed alternative.

6.6 Single points of failure

Current single points of failure include:

- One Express process.
- One MongoDB instance in local development.
- An in-memory notification queue.
- External Cloudinary availability for new image uploads.
- A single configured email provider.

For higher availability, deploy multiple stateless API instances behind a load balancer, use a MongoDB replica set, place jobs in a persistent distributed queue, use shared rate-limit storage, monitor external providers, and implement backup and restore procedures.

7. Technology and Design Decisions

7.1 Frontend

- Next.js 16.3.4 with App Router.
- React 19.
- TypeScript strict checking.
- Tailwind CSS 4 and shadcn/base UI primitives.
- TanStack Query for server state.
- React Hook Form and Zod for forms.
- Axios with credentials for API calls.
- Recharts for analytics.

7.2 Backend

- Node.js and Express 5.
- TypeScript 5.9.
- Mongoose 9 and MongoDB.
- Zod validation.
- JWT access tokens and opaque refresh tokens.
- bcrypt for PIN/password hashing.
- Pino structured logging.
- node-cron for scheduled jobs.
- Vitest, Supertest, and MongoDB Memory Server for testing.

7.3 Why MongoDB

MongoDB fits the implemented workload because:

- Issue records are naturally document-oriented.
- Status, category, timestamps, image URLs, and references are retrieved together.
- The schema can evolve as maintenance evidence and metadata mature.
- Mongoose provides validation, references, indexes, and a typed application model.
- Aggregation supports dashboard metrics.

MongoDB was not chosen because relationships do not exist. Users, halls, locations, issues, events, and assignments are related through object IDs. A relational database would also be valid, especially for stronger relational constraints and reporting joins. The trade-off accepted here is

application- and schema-level reference management in exchange for flexible document evolution and natural current-state retrieval.

7.4 Why external image storage

Storing images directly in MongoDB would enlarge backups and API memory use. Cloudinary handles binary storage and transformation. The application stores HTTPS URLs and metadata. The upload is signed by the backend and performed directly by the browser.

8. Data Design

8.1 Main collections

Users

Contains student/staff identity, email, password hash, role, active state, assigned halls, and optional allocated location.

Important uniqueness constraints:

- `studentId` sparse unique.
- `staffId` sparse unique.
- `email` unique.

Halls and locations

Halls provide controlled residence identity. Locations represent rooms and common areas. Issues reference hall and location rather than storing uncontrolled descriptions repeatedly.

Issues

Stores current operational state:

- Unique reference number.
- Reporter, hall, and location references.
- Category and reported/operational priority.
- Current status.
- Description and initial image URLs.
- Optional assigned personnel IDs.
- Submitted, resolved, closed, and dispute-window timestamps.

Issue events

Stores historical actions such as creation, acknowledgement, clarification, rejection, priority changes, comments, reopening, resolution, closure, and escalation.

Audit logs

Stores system-wide actor, action, resource, metadata, and timestamp records for administrative accountability.

Refresh and reset tokens

The database stores hashes rather than raw refresh and reset tokens, reducing exposure if those collections are read.

Reference counters

An atomic counter per hall code and date generates references such as:

HF-AKU-20260922-0001

8.2 Relationship overview

```
User 1 ---- * Issue (reporter)
Hall 1 ---- * Location
Hall 1 ---- * Issue
Location 1 ---- * Issue
Issue 1 ---- * IssueEvent
Issue * ---- * User (optional assignment)
User 1 ---- * RefreshToken
Issue 1 ---- * Notification
```

MongoDB does not enforce foreign keys in the same manner as a relational database. Integrity is protected through controlled service operations, Mongoose references, validation, authorization, and seed data.

8.3 Indexes

The issue model includes:

- Unique index on `referenceNumber` .
- `{ hallId, status, priority, createdAt }` for hall queues and sorting.
- `{ reporterId, createdAt }` for student history.
- `{ assignedToIds, status, updatedAt }` for optional personnel views.
- `{ category, hallId, createdAt }` for category/hall analysis.
- `{ status, priority, submittedAt }` for overdue and operational filtering.

Indexes convert many common lookups from collection scans toward logarithmic index lookup plus result retrieval. They also impose write and storage cost, so indexes are chosen around actual dashboard and queue access patterns.

8.4 Concurrent writes

Unique reference generation uses `findOneAndUpdate` with atomic `$inc` and a unique (`hallCode`, `date`) index. Concurrent submissions therefore increment one server-side counter rather than reading and writing a sequence in separate operations.

Single-document MongoDB updates are atomic. Multi-collection actions, such as updating an issue and adding event/audit records, are currently performed as separate writes. Production deployment should use a MongoDB replica set and transactions where all-or-nothing consistency is required.

8.5 Normalization and denormalization

The design is selectively normalized:

- Hall and location records are referenced to avoid inconsistent names.
- Current issue state is stored on the issue for fast dashboards.
- Historical changes are stored separately in issue events.
- Some user and hall data is populated for responses rather than copied permanently.

This is deliberate current-state/event-history separation, not accidental duplication.

8.6 Growth to 100 times the data

The first likely pressure points are:

1. Analytics functions that load candidate records into application memory for overdue and average-time calculations.
2. Global aggregation queries across all issues.
3. In-memory notifications and rate limiting.
4. Unbounded list endpoints if pagination is absent or insufficient.

Improvements would include:

- Move overdue computation into MongoDB aggregation or persist `targetResponseAt` with an index.
- Add pagination and bounded filters everywhere.
- Use pre-aggregated daily metrics for expensive dashboards.
- Archive old closed issues according to retention policy.
- Use Redis-backed queues, caching, and rate limiting.
- Examine query plans with `explain()` and production metrics before adding indexes.

9. API and Module Design

9.1 API base and documentation

The API is mounted at `/api/v1` . Swagger UI is served at `/api/v1/docs` . Local configuration currently uses backend port `5001` , though the configured `PORT` remains authoritative.

9.2 Important endpoint groups

Authentication

- `POST /auth/login`
- `POST /auth/logout`
- `POST /auth/refresh`
- `GET /auth/me`
- Password reset endpoints

Issues

- Create and list visible reports.
- Retrieve one permitted report and its event history.
- Add comments.
- Acknowledge, reject, or request clarification.
- Change operational priority.
- Optionally assign personnel.
- Change status according to policy.
- Reopen a resolved issue.

Residence directory

- List halls.
- List controlled hall locations.
- Obtain hall metrics.
- Update controlled metadata with administrative permission.

Administration

- Analytics overview.
- Audit log access.
- User listing and audited role changes.

9.3 Request validation

Zod schemas reject invalid or unknown request fields. JSON and URL-encoded payloads are limited to 10 KB. File binaries are not sent through the API; image uploads go directly to Cloudinary.

9.4 Error format

Errors use a stable structure with code, message, and request ID. The request ID links a client-visible failure to structured server logs.

9.5 Current contract limitation

The status request schema accepts an optional `message`, but `updateIssueStatus` currently does not persist that message in the event or audit record. Therefore, resolution-note functionality must not be presented as complete. The correct fix is to pass the message from controller to service, store it on the event and audit metadata, and add integration tests.

10. Frontend Design

10.1 Shared role-based route

Every authenticated user lands on `/dashboard`. The URL does not encode role. The authenticated user returned by `/auth/me` determines:

- Dashboard component.
- Navigation items.
- Available actions.
- Queries presented to the user.

The backend remains authoritative. Hiding a button is a usability measure, not a security boundary.

10.2 Role views

Student

- Own open/resolved metrics.
- Report issue action.
- Own issue list and details.
- Reopen action on a resolved issue.

Hall manager

- Assigned-hall queue.
- Pending, acknowledged, and resolved metrics.
- Acknowledge, reject, clarify, prioritize, resolve, and close actions permitted by policy.

Maintenance

- Assigned-hall work orders.
- Acknowledged, reopened, and other resolvable reports.
- Direct resolution action without individual assignment.

Administrator

- Cross-hall analytics.
- Audit logs.
- User and settings sections according to role.

10.3 Session guard

The dashboard layout waits for the current-session request. If no user is authenticated, it replaces the route with `/login`. This prevents the previous endless loading state for logged-out users.

10.4 State management

- TanStack Query manages API/server state and invalidation.
- React Hook Form manages form state.
- Zod validates client input.
- Local React or Zustand state handles interface concerns.

11. Algorithms and Complexity

HostelFix is primarily an information system rather than an algorithm-research project. Its core algorithms are state transition, unique reference allocation, access filtering, overdue detection, auto-close scanning, and aggregation.

11.1 State-transition validation

The policy stores a small list of valid `(from, to, predicate)` rules and searches for a matching transition.

If T is the number of transition rules, time complexity is $O(T)$ and space is $O(T)$. In this domain T is a small constant, so the operation is effectively constant time. A map keyed by `from:to` could provide expected $O(1)$ lookup, but it would not materially affect this workflow.

11.2 Unique reference allocation

Reference generation performs one indexed atomic upsert/update on `(hallCode, date)` and increments a sequence. With a B-tree index, lookup is approximately $O(\log C)$, where C is the number of counter documents; formatting is $O(1)$ for fixed-size fields.

The atomic increment is preferable to counting existing issues because counting is slower and creates race conditions under concurrent submissions.

11.3 Role-scoped issue retrieval

A student query filters by `reporterId`; hall staff filter by hall membership; administrators use an unrestricted query. With indexes, lookup is approximately $O(\log N + K)$, where N is total issues and K is returned issues. Without pagination, transferring and serializing K can still dominate.

11.4 Overdue calculation

The current analytics implementation fetches unresolved candidate issues and filters them in application memory. If u is the number of unresolved candidates, time is $O(u)$ and additional space is $O(u)$ for loaded documents. This is a known scaling bottleneck.

A better high-scale design would store indexed `targetResponseAt` or use an aggregation expression and query overdue records directly, reducing application memory and network transfer.

11.5 Auto-close job

The hourly job queries resolved issues whose dispute deadline is at or before now, then processes each result. For E expired issues, processing is $O(E)$ service calls after indexed candidate retrieval. Batch updates would be faster but would complicate per-issue events, audits, and notifications. The implementation prioritizes accountability over maximum batch throughput.

11.6 Analytics

MongoDB aggregation groups issues by status, priority, hall, and category. A grouping operation is generally $O(N)$ over matched records, subject to index-assisted filtering and database execution details. At high volume, pre-aggregation and time-bounded queries would be considered.

11.7 Performance evidence

No formal profiling, benchmark, or load-test report is currently present. Performance claims should therefore be framed as complexity analysis and index-aware design, not measured production

throughput. The first optimization effort should be driven by measurements of query latency, event-loop utilization, database explain plans, and memory consumption.

12. Security and Governance

12.1 Authentication

Login accepts University ID or email plus PIN/password. Password material is hashed with bcrypt. On successful authentication:

- A short-lived JWT access token is signed.
- A 64-byte random opaque refresh token is generated.
- Only a SHA-256 hash of the refresh token is stored.
- Both tokens are sent in HTTP-only cookies.

The browser does not store tokens in `localStorage` or `sessionStorage`, reducing exposure to JavaScript-based token theft.

12.2 Authorization

The system combines:

1. Authentication: who is the caller?
2. Role authorization: what category of action can that role perform?
3. Object-level authorization: does the caller own this issue or belong to the issue's hall?

The backend reloads the user on authenticated requests, so disabled accounts and updated role/hall assignments can take effect rather than relying only on stale JWT claims.

12.3 Token theft response

- Access tokens expire after a short duration.
- Refresh tokens are revocable database records.
- Refresh rotation deletes the old token before creating a new one.
- Logout deletes the presented refresh-token record.
- Password reset revokes all refresh tokens for that user.
- Production cookies should use `Secure` and an appropriate `SameSite` setting over HTTPS.

A remaining concern is that a stolen active access token remains usable until expiry unless a centralized revocation/version check is added. The short lifetime limits that window.

12.4 Threat model: leading attack vectors

Broken access control

Threat: A student requests another student's issue, or a hall manager accesses another hall.

Controls: Ownership/hall policies, role guards, backend query scoping, and negative permission tests.

Credential attacks

Threat: PIN brute force, credential stuffing, token theft, or account enumeration.

Controls: bcrypt hashing, login rate limiting, generic invalid-credential and password-reset responses, HTTP-only cookies, refresh rotation, and account active checks.

A five-digit PIN has limited entropy. Production should combine it with institutional identity assurance, lockout/monitoring, stronger secrets where possible, and potentially multi-factor authentication for privileged roles.

Injection and malicious input

Threat: Unexpected fields, oversized payloads, malformed IDs, or script content.

Controls: Strict Zod schemas, payload limits, Mongoose query construction, output encoding by React, Helmet headers, and centralized errors. A dedicated content-security-policy review and security test should still be completed.

Malicious or privacy-invasive images

Threat: Invalid URLs, oversized uploads, sensitive room images, or retained personal information.

Controls: Signed direct upload, Cloudinary URL validation, external binary storage, and authorization over issue records. File-type/size policy, private delivery configuration, retention, moderation, and deletion workflows remain production requirements.

12.5 OWASP alignment

Examples include:

- **A01 Broken Access Control:** object-level policies and permission-denial tests.
- **A02 Cryptographic Failures:** bcrypt password hashing, token hashing, and HTTPS/Secure-cookie production requirements.
- **A03 Injection:** strict Zod schemas and controlled query construction.
- **A05 Security Misconfiguration:** environment validation, Helmet, restricted CORS, and production cookie settings.

- **A07 Identification and Authentication Failures:** rate limits, generic errors, short-lived access tokens, refresh rotation, and revocation.
- **A09 Security Logging and Monitoring Failures:** request IDs, Pino logs, issue events, and audit logs.

The project follows relevant OWASP principles but has not undergone a formal OWASP verification or penetration test.

12.6 Data at rest and in transit

In transit, production must use HTTPS. At rest, password and token material is hashed, but application records are not field-level encrypted by the code. Production should use encrypted MongoDB storage/Atlas encryption, encrypted backups, restricted database network access, and managed secrets.

12.7 Database credential leakage

If a database credential leaks, the blast radius depends on its privileges and network reachability. Controls should include:

- A least-privilege application database user limited to the HostelFix database.
- Network allowlisting/private networking.
- Separate development, test, staging, and production credentials.
- Secret-manager storage rather than committed files.
- Credential rotation and audit monitoring.
- Encrypted backups and tested recovery.

No secret should be included in this report or version control.

12.8 Governance

Before institutional launch, policy owners must approve:

- Data retention durations.
- Image access and deletion.
- Role provisioning and periodic review.
- Emergency escalation contacts.
- Audit-log access.
- Student dispute handling.
- Contractor identification.
- Incident-response and breach-notification procedures.

13. Software Engineering Process

13.1 Development approach

The implementation reflects iterative and risk-driven development rather than a documented formal Scrum process. The workflow was revised after critical operational review, then backend policies, tests, frontend views, migration, and documentation were updated together.

It is defensible to describe the process as iterative Agile-inspired development using short feedback cycles, but not to claim formal sprints, ceremonies, velocity, or product-owner sign-off unless separate evidence exists.

13.2 Code organization

The backend is organized by domain modules, while shared middleware and policies remain explicit. The frontend separates app routes, dashboard components, hooks, API utilities, providers, forms, types, and constants. This supports maintainability because a new developer can trace an issue action from route to validation, controller, service, policy, model, and test.

13.3 Version control

The provided project root is not currently recognized as a Git repository. Therefore, this document cannot honestly claim a branching strategy, commit history, pull-request review, or CI pipeline based on repository evidence. Before final submission, the project should be placed under Git version control with:

- A protected main branch.
- Short-lived feature branches.
- Reviewed pull requests.
- Concise commits tied to requirements or defects.
- Automated test, typecheck, lint, and build checks.

If a separate Git repository exists elsewhere, its history should be included as defense evidence.

13.4 Static analysis and quality gates

The project uses:

- TypeScript compilation/typecheck.
- ESLint.
- Next.js production build.
- Vitest unit/integration/permission tests.

- OpenAPI YAML parsing and runtime Swagger UI.

A future CI workflow should run these checks on each pull request.

14. Testing and Evaluation

14.1 Testing strategy

Unit testing

Tests isolated policy decisions and reference-number behavior.

Integration testing

Tests exercised Express endpoints with Supertest and MongoDB Memory Server, including authentication, halls, issue lifecycle, and automatic closure.

Permission testing

Negative tests verify that:

- One student cannot read another student's issue.
- A manager cannot access another hall's data.
- Maintenance cannot self-assign arbitrary work.
- Non-system administrators cannot escalate roles.

Frontend testing

The current frontend unit suite validates status behavior. The project also passes TypeScript checking and production build. Broader component and end-to-end tests remain limited.

14.2 Current verified results

At the last recorded verification:

- Backend: 10 test files, 30 tests passing.
- Backend typecheck, lint, and build passed in the completed workflow implementation.
- Frontend unit tests: 2 passing.
- Frontend typecheck passed.
- Frontend production build passed.
- Frontend lint passed with two non-blocking raw-image warnings.
- Seeded logins for all five roles were tested against the API and returned the expected roles.

14.3 Example failure and correction

The test infrastructure originally used an older email/password assumption even though authentication had changed to ID/PIN. This caused drift between tests and the actual API. Test helpers were updated to use the real authentication contract. Test parallelism against shared in-memory state and low rate-limit values also caused avoidable failures; the suite configuration and helpers were corrected.

Another integration defect was stale persisted issue status `verified` after the new enum removed it. A migration mapped legacy `verified` records to `acknowledged`.

14.4 Correctness versus “it runs”

Correctness is evaluated through:

- State-transition tests.
- Role/ownership/hall permission tests.
- Auto-close behavior.
- Authentication role responses.
- Type checking and schema validation.
- Production frontend compilation.

However, operational effectiveness requires more than software correctness. A pilot should measure acknowledgement time, resolution time, reopening rate, report loss, user satisfaction, and staff adoption.

14.5 Missing evaluation

The following are not currently evidenced:

- Formal load testing.
- Measured test coverage percentage.
- Penetration testing.
- Accessibility audit.
- Cross-browser matrix.
- Independent user-acceptance testing.
- Quantified before/after operational outcomes.

These should be presented as future evaluation, not invented results.

15. Deployment and Operations

15.1 Local runtime

- Frontend: `http://localhost:3000`
- Backend: configured locally as `http://localhost:5001`
- MongoDB: `mongodb://127.0.0.1:27017/hostelfix`

The backend must have MongoDB running before startup. The development database can be seeded with the provided scripts. Seeded accounts use PIN 12345 only for local demonstration and must never be used in production.

15.2 Production topology

A reasonable production design is:

- Next.js deployed behind HTTPS/CDN.
- Express API in containers or managed compute behind a load balancer.
- MongoDB replica set or managed MongoDB service.
- Redis/BullMQ or managed queue for durable jobs.
- Cloudinary configured with signed uploads and an approved retention policy.
- Managed secret storage.
- Centralized logs, metrics, alerts, and error tracking.
- Automated backup and restore testing.

15.3 Scaling to ten times the load

1. Horizontally scale stateless frontend and API processes.
2. Move queues and rate limits from memory to Redis-backed infrastructure.
3. Verify MongoDB indexes and query plans.
4. Add pagination and cache suitable read-heavy reference data.
5. Pre-aggregate expensive analytics.
6. Separate notification workers if workload justifies it.

The system should remain a modular monolith until measured constraints justify service extraction.

16. Limitations and Future Work

16.1 Biggest current limitation

The largest product-level limitation is the absence of documented real-user validation and pilot evidence. The architecture can be technically correct while still failing adoption. The project's central claim is reduced operational friction, so that claim should eventually be measured with actual users.

16.2 Important technical limitations

1. Status message is currently not persisted by the status service.
2. Duplicate reports cannot be formally linked.
3. External resolver identity is not a structured field.
4. Completion photos are not separately supported.
5. Emergency-contact UI is absent.
6. In-memory jobs and rate limits do not support reliable multi-instance operation.
7. Some analytics load records into application memory.
8. Multi-collection writes are not consistently transactional.
9. Frontend end-to-end and accessibility coverage is limited.
10. No load, penetration, or user-acceptance report exists.
11. The current seed contains ten halls while original project scope states nine.
12. The server currently begins listening before database connection completes; startup ordering should be improved so database failure causes a clear fail-fast exit rather than an unhandled rejection while a port may already be open.

16.3 Prioritized roadmap

Immediate correctness

- Persist resolution/status messages and test them.
- Correct hall-scope documentation/code discrepancy.
- Make backend startup connect to MongoDB before listening.
- Add pagination to list endpoints.

Operational completeness

- Add duplicate linking and subscriber notifications.
- Add structured resolver and completion evidence.
- Add approved emergency contacts.
- Add role-provisioning and retention workflows.

Production readiness

- Persistent queue and shared rate limiter.
- Replica set and transaction review.
- Secrets manager, backups, monitoring, and alerts.
- CI/CD and Git-based review process.
- Load, security, accessibility, and disaster-recovery testing.

Research and evaluation

- Stakeholder interviews and baseline survey.
- Pilot deployment in one hall.
- Measure acknowledgement, resolution, reopening, and adoption.
- Refine the interface based on observed usage.

17. Critical Reflection and Intellectual Contribution

17.1 What would be done differently

If starting again, stakeholder validation would happen before finalizing workflow and schema. A prototype of the reporting and acknowledgement experience would be tested with students, hall staff, and maintenance workers. This likely would have produced the simplified workflow earlier and reduced migration effort.

The implementation would also begin with Git and CI from the first commit, explicit pagination, a replica-set development environment for transactions, and a clear distinction between initial report evidence and completion evidence.

17.2 Most defensible design decision

The strongest decision is making acknowledgement and assignment non-blocking. It recognizes that institutional work is socio-technical: software should improve visibility without demanding that all behavior conform to the application.

17.3 Least certain design decision

MongoDB is defensible but not inevitable. The domain has meaningful relationships, audit requirements, and reporting needs that a relational database could model strongly. MongoDB remains reasonable because current-state issue documents and evolving evidence fit document storage, while indexes and controlled references support the present queries. The decision should be revisited if reporting joins, strict relational integrity, or transactional complexity dominate.

17.4 Intellectual contribution

The contribution is not the invention of login, REST, MongoDB, or dashboards. It is the combination of:

- A workflow derived from adoption constraints.
- Separation of physical work from digital accountability.

- Current-state plus immutable-history modeling.
- Role plus object-level authorization.
- A dispute window that replaces manual closure bottlenecks.
- A shared route whose content and data scope adapt to authenticated role.

This demonstrates computing mastery through requirements analysis, state modeling, secure API design, concurrency-aware reference generation, database indexing, modular architecture, asynchronous processing, testing, and critical trade-off analysis.

18. Oral-Defense Question Bank and Model Answers

18.1 Problem Definition and Motivation

Q: What specific problem does this solve, and what evidence shows it is real?

Answer: HostelFix addresses fragmented maintenance reporting and weak follow-up across verbal reports, messaging, logbooks, and informal handoffs. It creates one scoped record of what is broken, whether hall staff know, whether it was fixed, and whether the student disputes the result. The design is grounded in operational workflow analysis, particularly the recognition that repairs often begin outside an application. I must be precise that the current repository does not contain formal survey or interview data; a hall pilot and stakeholder study are required to quantify the problem and evaluate adoption.

Q: Who are the users and how was the problem validated?

Answer: The direct users are student residents, hall managers/caretakers, university maintenance personnel, university administrators, and system administrators. Requirements were validated through role and use-case analysis and iterative critique of the proposed workflow. Formal validation artifacts such as interview transcripts and survey results are not currently present, so I would not claim them. My next evaluation step would be interviews and a pilot in one hall.

Q: What existing solutions were reviewed, and what do they fail to do?

Answer: The practical alternatives are verbal reporting, WhatsApp/telephone, logbooks, spreadsheets, and generic ticketing systems. Informal methods are fast but weak in traceability and analytics. Generic ticketing tools are traceable but can impose assignment-heavy workflows that do not match hall maintenance. HostelFix keeps the informal channel available while adding hall scope, resident ownership, audit events, a dispute window, and cross-hall metrics.

Q: Why is this a computer science or IT problem?

Answer: Software cannot repair a pipe, but it can solve identity, authorization, state consistency, concurrent references, notification, audit, and analytics problems across multiple roles. Those require secure session management, data modeling, state-machine policy, asynchronous jobs, indexing, and tested APIs.

Q: State the novelty in one sentence.

Answer: HostelFix is a hall-scoped maintenance record that preserves accountability without making digital assignment and progress updates prerequisites for physical work.

18.2 Requirements Engineering

Q: How were requirements derived?

Answer: Through stakeholder-role analysis, use cases, workflow modeling, permission-matrix analysis, and iterative critique. A major example is replacing the original six-stage lifecycle after identifying that it would create an adoption bottleneck. Formal stakeholder research is a remaining methodological task.

Q: What was deliberately scoped out?

Answer: Public registration, inventory and procurement, payment, GPS tracking, AI diagnosis, emergency dispatch, contractor management, and microservices. These were excluded to keep the first release focused on reporting, awareness, resolution, dispute, and accountability.

Q: How do requirements trace to deliverables?

Answer: Each sensitive requirement maps to frontend routes, API endpoints, policy functions, data fields, and tests. Student isolation, for example, maps to `/dashboard/issues`, reporter-scoped queries, `canViewIssue`, and a negative permission test.

Q: How would you handle stakeholder/technical conflict?

Answer: Identify the underlying need and separate it from the proposed mechanism. Mandatory assignment was requested for accountability, but it conflicted with low-friction operations. I retained assignment as optional metadata and preserved actor audit events while removing it as a transition gate.

18.3 System Design and Architecture

Q: Walk through the architecture.

Answer: A Next.js client calls an Express REST API using credentialed HTTPS requests. Express owns authentication, authorization, validation, workflow, analytics, uploads, audit, and jobs. MongoDB stores operational data; Cloudinary stores image binaries; an email adapter handles notification delivery. The backend is a modular monolith organized by domain.

Q: Why a modular monolith rather than microservices?

Answer: The current scale and team do not justify distributed-system overhead. A modular monolith is easier to deploy, debug, test, and keep consistent. Module boundaries preserve the option to extract notifications or analytics later when measured load or team ownership justifies it.

Q: What are the single points of failure?

Answer: One API process, one local MongoDB process, an in-memory queue, and external image/email providers. Production mitigation includes multiple API instances, a replica set, persistent queue, shared rate limiter, monitoring, backups, and provider failure handling.

Q: How do components communicate and why REST?

Answer: The browser and frontend communicate with the API through JSON REST endpoints. REST is widely supported, easy to document with OpenAPI, and sufficient for resource-oriented operations. Jobs are decoupled through an internal queue. gRPC would add complexity without a demonstrated latency or strongly typed service-to-service requirement.

Q: What patterns were used?

Answer: Layered modular architecture, service layer, policy/strategy-style authorization functions, repository-like Mongoose models, adapter-based external services, current-state plus event-history modeling, and asynchronous job processing. Each exists for a concrete reason: policy functions centralize access decisions, services centralize workflows, and event history preserves accountability without slowing dashboard queries.

18.4 Data Design

Q: Explain the schema.

Answer: Users reference assigned halls; halls contain controlled locations; issues reference reporter, hall, and location; issue events reference issues and actors; assignment is optional many-to-many metadata; notification and audit records preserve delivery and administrative history. Current state is stored on issues and historical transitions in issue events.

Q: What keys and indexes matter?

Answer: MongoDB ObjectIds are primary identifiers. Unique keys exist for user email/student/staff ID and issue reference number. Compound issue indexes support hall queues, student history, assignment views, category analytics, and overdue filtering. The counter has a unique hall/date index for concurrent reference allocation.

Q: How are integrity and concurrency handled?

Answer: Strict validation, controlled services, unique indexes, atomic single-document updates, and an atomic `$inc` counter handle common integrity and concurrency needs. Cross-collection writes are separate today; production should use replica-set transactions where all writes must succeed or fail together.

Q: What breaks at 100 times the data?

Answer: In-memory overdue and average-time calculations, global aggregations, unbounded result sets, and in-memory jobs/rate limiting. I would add pagination, database-side deadlines, pre-aggregated metrics, archive policies, durable queues, shared rate limiting, and query-plan measurement.

Q: Why NoSQL?

Answer: Issue current state is naturally document-shaped, the evidence schema may evolve, and MongoDB aggregation supports dashboards. Controlled references preserve relationships. A relational database is a valid alternative and may be stronger if complex joins and strict relational constraints become dominant.

18.5 Algorithms and Complexity

Q: What are the core algorithms?

Answer: State-transition lookup, atomic reference generation, role-scoped query construction, overdue filtering, auto-close selection, and analytics aggregation. Transition lookup is $O(T)$ over a small constant rule set. Indexed retrieval is approximately $O(\log N + K)$. Current overdue calculation is $O(U)$ time and space over unresolved candidates, which is the clearest optimization target.

Q: Why atomic counter generation?

Answer: Counting existing issues and adding one creates a race under concurrent submissions. An indexed atomic `$inc` provides a unique sequence without a client-side race.

Q: Where is the performance bottleneck and how do you know?

Answer: Code analysis identifies analytics functions that load candidate documents into application memory. I do not yet have profiling or benchmark evidence, so I describe it as a likely bottleneck rather than a measured one. Production optimization should begin with telemetry and database explain plans.

Q: What would you optimize first?

Answer: Persist/index `targetResponseAt` , paginate issue lists, move overdue calculations into MongoDB, and replace in-memory jobs/rate limits. Those changes reduce memory and support horizontal scaling.

18.6 Security and Governance

Q: How do authentication and authorization work?

Answer: ID/email and PIN/password are checked against a bcrypt hash. The API issues an HTTP-only JWT access cookie and opaque refresh cookie. Each protected request verifies the token, reloads the active user, then applies role plus object-level ownership or assigned-hall policy.

Q: What if a token is stolen?

Answer: HTTP-only storage reduces script access, access tokens are short-lived, refresh tokens are hashed and rotated, logout revokes the current refresh token, and password reset revokes all refresh tokens. A stolen access token can remain valid until expiry, so HTTPS, Secure cookies, short TTL, monitoring, and potentially token versioning are important.

Q: What are the top three attacks?

Answer: Broken object-level authorization, credential/token attacks, and malicious input or uploads. They are addressed through scoped policies and negative tests; bcrypt, rate limits, cookie controls, rotation, and generic errors; and strict schemas, payload limits, Helmet, Cloudinary signing, and URL validation.

Q: How is data protected?

Answer: Passwords and stored tokens are hashed. Production traffic must use HTTPS and Secure cookies. Database storage and backups should use provider encryption and least-privilege credentials. The application does not currently implement field-level encryption for issue content.

Q: Which standard was followed?

Answer: The controls align with OWASP Top 10 categories, especially broken access control, authentication failures, injection, security misconfiguration, and security logging. No formal certification or penetration test has been completed.

Q: What if database credentials leak?

Answer: The impact should be constrained through a database-specific least-privilege user, private networking, environment separation, secret management, monitoring, and rapid rotation. If an unrestricted credential is used, the blast radius could include all application records, so credential scope is a production governance requirement.

18.7 Software Engineering Process

Q: What methodology was followed?

Answer: Iterative, risk-driven development with Agile-inspired feedback cycles. The workflow was revised after operational critique, and code, tests, migration, UI, and documentation were updated together. I would not claim formal Scrum ceremonies because that evidence is not present.

Q: How was version control managed?

Answer: The current project root is not recognized as a Git repository, so I cannot substantiate a branching or commit strategy from this copy. Before submission or team continuation, I would initialize or restore Git history, use protected main, feature branches, pull-request review, and CI quality gates.

Q: What testing was performed?

Answer: Backend unit, integration, and permission-denial tests; frontend unit tests; TypeScript checks; lint; production builds; API login smoke tests; and browser route checks. Formal load, penetration, accessibility, and user-acceptance testing remain outstanding.

Q: Was there code review or CI/CD?

Answer: Static analysis and automated local quality gates exist, but the repository evidence does not show a CI pipeline or formal peer-review record. A pull-request pipeline should run tests, typecheck, lint, build, dependency scanning, and OpenAPI validation.

Q: How is maintainability supported?

Answer: Domain modules own routes, validation, controllers, services, and models; reusable policies centralize authorization; the frontend separates routes, components, hooks, providers, and types; and OpenAPI documents the external contract.

18.8 AI/ML Questions

Q: What AI model did the system use?

Answer: None. HostelFix is a transactional information system. Adding AI simply to appear advanced would introduce unsupported predictions and new privacy risks. A future classifier could suggest category or priority, but hall management would still need control, and it would require representative labeled data, bias analysis, and measured benefit.

Q: How would you evaluate a future model?

Answer: Use a time- and hall-aware train/validation/test split to prevent leakage, evaluate per-class precision, recall, F1, and calibration, inspect errors by hall/category, and retain human override. No such model or dataset exists in the current system.

18.9 Testing and Evaluation

Q: How do you know the system meets requirements?

Answer: Requirements map to routes, policies, models, UI, and automated tests. Sensitive isolation is tested negatively, not just through successful paths. Builds and typechecks establish structural correctness. Operational effectiveness still requires user and pilot metrics.

Q: Give an initially failing case and fix.

Answer: Tests used an old email/password authentication contract while the API used ID/PIN. Helpers were updated to authenticate through the real contract. A second case involved legacy `verified` database records after status simplification; a migration mapped them to `acknowledged`.

Q: Was load testing performed?

Answer: No formal load test is documented. I can explain complexity and indexes, but I should not invent throughput results. A suitable next test would use k6 or Artillery against login, issue creation, hall queues, and analytics with realistic concurrency.

Q: Who tested it besides the developer?

Answer: No independent UAT evidence is stored in the repository. That is a limitation. A supervised pilot with students, hall managers, and maintenance workers should produce signed scenarios, observations, and change requests.

18.10 Critical Reflection

Q: What is the single biggest limitation?

Answer: Lack of formal real-user validation. The system's defining claim is reduced workflow friction, so adoption and operational impact must be measured in a pilot rather than inferred only from code.

Q: What would you do differently?

Answer: Validate workflows with users before implementation, establish Git and CI immediately, model completion evidence separately, build pagination from the start, and use a replica-set development environment for transaction testing.

Q: What did you learn technically?

Answer: The most important lesson is that technically detailed workflows can still be poor systems. Good architecture includes socio-technical fit. I also learned to combine role and object-level authorization, model current state separately from immutable history, handle concurrent counters atomically, and treat background delivery as independent from core persistence.

Q: Defend a decision you are least confident about.

Answer: MongoDB is the least exclusive decision because PostgreSQL would also fit. I defend it through document-shaped issue state, evolving evidence, Mongoose validation, and existing aggregation/index support. I also acknowledge its weaker built-in relational constraints and would revisit the choice if reporting joins and transactional relationships become dominant.

Q: What did you add rather than assemble?

Answer: The intellectual work lies in the workflow and policy design: eliminating assignment gates while preserving accountability, adding a dispute window and auto-close, defining hall-scoped object authorization, and separating present state from immutable events. Frameworks supplied tools; the domain model and trade-offs determine system behavior.

Q: How does this demonstrate CS/IT mastery?

Answer: It integrates requirements engineering, state-machine design, secure authentication, object-level authorization, database modeling and indexing, concurrency control, REST API design, asynchronous jobs, frontend state management, testing, observability, and critical analysis of scalability and governance.

19. Demonstration Script

A concise defense demonstration should use separate browser sessions or log out between roles.

19.1 Preparation

1. Start MongoDB.
2. Start the backend on the configured port.
3. Seed development users.
4. Start the frontend.
5. Confirm `/health` and `/api/v1/docs` .

Development accounts all use PIN 12345 :

ID	Role
11287773	Student
10000001	Hall manager
10000002	Maintenance
10000003	University administrator
10000004	System administrator

These credentials are demonstration-only.

19.2 Main scenario

1. Log in as the student and show `/dashboard` .
2. Create a plumbing issue with hall, room, priority, description, and optional image.
3. Show the generated reference and `submitted` state.
4. Log in as hall manager and show role-specific content at the same `/dashboard` URL.
5. Acknowledge the issue and show its event history.
6. Log in as maintenance and show the assigned-hall work-order view.
7. Mark the issue resolved without individual assignment or `in_progress` .
8. Log in as the student and show the dispute/reopen option.
9. Explain that an hourly job auto-closes after the configured window.
10. Log in as administrator and show analytics and audit logs.

19.3 Security scenario

- Attempt to access another student's issue and explain the expected `403` .
- Explain hall-manager isolation.
- Show HTTP-only cookies rather than local-storage tokens.
- Show OpenAPI and request-ID error structure.

19.4 Reflection during demonstration

Explicitly state that maintenance work may happen before acknowledgement. The digital state records the outcome; it does not prevent a caretaker or technician from acting.

20. Conclusion

HostelFix demonstrates that maintenance software should be designed around actual institutional behavior. The system provides role-scoped reporting, controlled status transitions, auditability, notifications, analytics, and a dispute mechanism while avoiding the central failure of over-digitization: making application updates a prerequisite for physical work.

The implemented modular architecture, object-level authorization, indexed MongoDB model, atomic reference generation, and automated tests provide a strong technical foundation. At the same time, the project is honest about its unfinished work. Production deployment requires formal stakeholder validation, persistent infrastructure, stronger operational governance, completion evidence, duplicate handling, performance testing, security assessment, and resolution of the approved-hall discrepancy.

The project's main contribution is therefore both technical and analytical: it shows how a system can increase accountability without becoming the bottleneck it was intended to remove.

21. References and Evidence Map

21.1 Internal project evidence

- `README.md` — project overview and operating model.
- `backend/README.md` — backend setup and implementation summary.
- `frontend/README.md` — frontend setup and shared dashboard architecture.
- `hostelfix-backend-docs/workflows.md` — operational lifecycle.
- `hostelfix-backend-docs/data-model.md` — collection design and indexes.
- `hostelfix-backend-docs/api-contract.md` — API contract.
- `hostelfix-backend-docs/technical-decisions.md` — architecture rationale.
- `hostelfix-backend-docs/adr-001-simplified-workflow.md` — accepted workflow decision.
- `backend/docs/openapi.yaml` — canonical machine-readable API description.
- `backend/src/policies/issue.policy.ts` — state and authorization policy.
- `backend/src/modules/issues/issue.service.ts` — issue workflow implementation.
- `backend/src/modules/auth/auth.service.ts` — authentication and token lifecycle.
- `backend/src/jobs/auto-close-resolved-issues.job.ts` — automatic closure.
- `backend/tests/` — unit, integration, and permission evidence.

21.2 External frameworks to cite in a formal academic bibliography

The final institution-formatted document should add properly formatted references for:

- OWASP Top 10 and OWASP Application Security Verification Standard.
- MongoDB and Mongoose documentation.
- Express documentation.
- Next.js documentation.
- JSON Web Token standards and security guidance.
- ISO/IEC 25010 software quality model, if used for evaluation.
- Relevant maintenance-management and information-systems literature.
- University of Ghana residence and data-governance policies, where officially available.

Do not claim that a standard was formally certified or that an empirical study was conducted unless evidence is attached.

Appendix A: Short Defense Opening

HostelFix is a role-based maintenance reporting and tracking platform for University of Ghana halls. Its goal is not to digitize every physical handoff. Its goal is to create a reliable record of what is broken, whether the responsible hall knows, whether it was fixed, and whether the resident disputes the result. I implemented it as a Next.js client, an Express and TypeScript modular API, and MongoDB. The key design decision was simplifying the workflow to Submitted, Acknowledged, Resolved, and Closed, with rejection and reopening. Assignment remains optional, so maintenance work can continue through the practical channels already used in the halls. The system adds object-level authorization, audit events, analytics, notifications, and automatic closure after a dispute window without turning the software into an operational bottleneck.

Appendix B: One-Minute Architecture Answer

The browser renders a Next.js frontend and calls an Express REST API using HTTP-only authentication cookies. Express owns all domain and permission rules. It is organized as a modular monolith with authentication, users, residences, issues, assignments, notifications, analytics, audit logs, and upload modules. MongoDB stores current issue state and related records, while issue events preserve history. Cloudinary stores images. Scheduled jobs escalate overdue issues and auto-close resolved issues after the dispute window. I chose a modular monolith because the current scale does not justify microservice complexity, but module boundaries leave room to extract notifications or analytics later.

Appendix C: One-Minute Limitation Answer

The biggest limitation is not a missing framework feature; it is the absence of formal user and pilot validation. The system is designed to reduce workflow friction, but that must be measured with students, hall staff, and maintenance workers. Technically, the most immediate defects are that status messages are accepted but not persisted, duplicate reports are not linked, completion evidence is not modeled, the queue is in-memory, and the current seed contains ten halls while the original scope states nine. I would correct those before production and then run load, security, accessibility, and user-acceptance testing.